Security requirement

Secure Shell (SSH)

Deutsche Telekom Group

Version	5.0
Date	Dec 1, 2025
Status	Released

Publication Details

Published by
Deutsche Telekom AG
Vorstandsbereich Technology & Innovation
Chief Security Officer

Reuterstrasse 65, 53315 Bonn
Germany

File name	Document number	Document type
	3.04	Security requirement

Version	State	Status
5.0	Dec 1, 2025	Released

Contact	Validity	Released by
Telekom Security psa.telekom.de Mirko Rascher mirko.rascher@telekom.de	Dec 1, 2025 - Nov 30, 2030	Stefan Pütz, Leiter SEC-T-TST

Summary
Security requirements for SSH server, SFTP server and the SSH protocol.

Table of Contents

1. Introduction

SSH (Secure Shell) is a client/server application inclusive a network protocol that can be used to access systems on terminal level and to transfer data. As SSH is typically used for management access, this service has a high security demand. This document includes security requirements for SSH server, SFTP server and the SSH protocol.

It is recommended to use OpenSSH as this is a well-known solution with a huge developer community. Accordingly requirements are aligned to OpenSSH.

This security document has been prepared based on the general security policies of the group in correlation with security best practices and vendor recommendations. The security requirement is used as a basis for an approval in the PSA process, among other things. It also serves as an implementation standard for units which do not participate in the PSA process. These requirements shall be taken into account from the very beginning, including during the planning and decision-making processes. When implementing these security requirements, the precedence of national, international and supranational law shall be observed.

2. sshd_config

Req 1	The version 2 of the SSH protocol must be used. (Automatically compliant when using OpenSSH 7.4+).

User roles: Operation, Development, Integration

SSHv1 must permanently be disabled in configuration of the SSH server. With OpenSSH 7.4 support for SSHv1 completely removed and need not be configured explicitly.

Reference:
[TR-02102-4] BSI-Technical Guideline: Cryptographic Mechanisms - Recommendations and Key Lengths - Part 4 SSH, Version 2025-01

Motivation: The SSH version 1 has cryptographic weaknesses and is obsolete today. With the use of SSHv1 the confidentiality and integrity of transmitted data cannot be guaranteed.

Implementation example: OpenSSH Parameter: Protocol 2

Compliance check: $ ssh -V OpenSSH_8.2p1 Ubuntu-4ubuntu0.1, OpenSSL 1.1.1f 31 Mar 2020

oder

$ ssh -Q protocol-version 2

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Disruption of availability
- Unnoticeable feasible attacks

ID: 3.50-46/8.0

Req 2	SSH MaxStartups must be set to "10:30:100" or less. (OpenSSH default)

The MaxStartups parameter specifies the maximum number of concurrent unauthenticated connections to the SSH daemon. This value must be "`10:30:100`" or less, this is OpenSSH default.

Motivation: To protect a system from denial of service due to a large number of pending authentication connection attempts, use the rate limiting function of MaxStartups to protect availability of sshd logins and prevent overwhelming the daemon.

Implementation example: `sshd_config` contains:
```
MaxStartups 10:30:100
```

Compliance check:
```
$ sshd -T | grep -i MaxStartups
```

For this requirement the following threats are relevant:
- Disruption of availability

ID: 3.04-2/5.0

Req 3	Only the following key exchange algorithms must be used: curve448-sha512, sntrup761x25519-sha512@openssh.com, curve25519-sha256@libssh.org, curve25519-sha256, ecdh-sha2-nistp521, ecdh-sha2-nistp384, cdh-sha2-nistp256, diffie-hellman-group18-sha512, diffie-hellman-group16-sha512, diffie-hellman-group-exchange-sha256 or mlkem768x25519-sha256@openssh.com.

User roles: Operation, Development, Integration

For key exchange the following algorithms are allowed:

Algorithm	Reference specifications
curve448-sha512	[RFC-7748]
sntrup761x25519-sha512@openssh.com	
curve25519-sha256@libssh.org	[RFC-7748]
curve25519-sha256	[RFC-7748]
ecdh-sha2-nistp521	[RFC-5656]
ecdh-sha2-nistp384	[RFC-5656]
ecdh-sha2-nistp256	[RFC-5656]
diffie-hellman-group18-sha512	[RFC-8268]
diffie-hellman-group16-sha512	[RFC-8268]
diffie-hellman-group-exchange-sha256	[RFC-4419]
mlkem768x25519-sha256@openssh.com	

Remark on curve448 and curve25519:
curve448 und curve25519 are not (explicitly) recommended by BSI, but no weaknesses are known so far, and therefore they are to be classified as secure.

References:
[RFC-4419] Diffie-Hellman Group Exchange for SSH
[RFC-5656] Elliptic Curve Algorithm Integration
[RFC-7748] Elliptic Curves for Security
[RFC-8268] More MODP Diffie-Hellman Groups
[RFC-8731] Secure Shell (SSH) Key Exchange Method Using Curve25519 and Curve448
[TR-02102-4] BSI-Technical Guideline: Cryptographic Mechanisms - Recommendations and Key Lengths - Part 4 SSH, Version 2025-01

Motivation: An attacker can possibly break the encryption of transported data if weak ciphers and algorithms are used to access secret data.

Implementation example: sshd_config contains:
```
KexAlgorithms curve448-sha512, curve25519-sha256@libssh.org, curve25519-sha256, ecdh-sha2-nistp521, ecdh-sha2-nistp384, ecdh-sha2nistp256, diffie-hellman-group18-sha512, diffie-hellman-group16sha512, diffie-hellman-groupexchange-sha256
```

Compliance check: $ sshd -T | grep -i KexAlgorithms

For this requirement the following threats are relevant:
- Unauthorized access or tapping of data

ID: 3.50-48/8.0

Req 4	Only the following ciphers must be used: aes256-gcm@openssh.com, aes128-gcm@openssh.com, chacha20-poly1305@openssh.com, aes256-ctr, aes192-ctr or aes128-ctr.

User roles: Operation, Development, Integration

Outdated and insecure ciphers and algorithms must not be used. Use the following ciphers for SSH:

Cipher	Reference specifications
aes256-gcm@openssh.com	[RFC-5647]
aes128-gcm@openssh.com	[RFC-5647]
chacha20-poly1305@openssh.com	[RFC-8439]
aes256-ctr	[RFC-4344]
aes192-ctr	[RFC-4344]
aes128-ctr	[RFC-4344]

References:
[RFC-4344] The Secure Shell (SSH) Transport Layer Encryption Modes
[RFC-5647] AES Galois Counter Mode for the Secure Shell Transport Layer Protocol
[RFC-8439] ChaCha20 and Poly1305 for IETF Protocols

Motivation: An attacker can possibly break the encryption of transported data if weak ciphers and algorithms are used to access secret data.

Implementation example: sshd_config contains:

```
Ciphers aes256-gcm@openssh.com,aes128-gcm@openssh.com,chacha20-poly1305@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr
```

Compliance check: $ sshd -T | grep -i Ciphers

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities

ID: 3.50-49/8.0

Req 5	Only the following MAC algorithms must be used: hmac-sha2-512-etm@openssh.com, hmac-sha2-512, hmac-sha2-256-etm@openssh.com or hmac-sha2-256.

User roles: Operation, Development, Integration

It is important to avoid the use of insecure MAC algorithms for SSH. Examples of such outdated algorithms are MD5 and SHA1. The following MAC algorithms are allowed and must be configured for SSH daemon:

Algorithm	Reference specifications
hmac-sha2-512-etm@openssh.com	[RFC-6688]
hmac-sha2-512	[RFC-6688]
hmac-sha2-256-etm@openssh.com	[RFC-6688]

hmac-sha2-256	[RFC-6688]

References:
[RFC-6688] SHA-2 Data Integrity Verification for the Secure Shell (SSH) Transport Layer Protocol

Motivation: An attacker can possibly break the encryption of transported data if weak ciphers and algorithms are used to access secret data.

Implementation example: sshd_config contains:

```
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-512,hmac-sha2-256-
etm@openssh.com,hmac-sha2-256
```

Compliance check: $ sshd -T | grep -i MACs

For this requirement the following threats are relevant:
- Unauthorized modification of data

ID: 3.50-50/8.0

Req 6	Only the following Host Key Algorithms (a.k.a. public key signature algorithms or server authentication algorithms) must be used: ssh-ed448, ssh-ed25519, ssh-ed25519-cert-v01@openssh.com, sk-ssh-ed25519@openssh.com, sk-ssh-ed25519-cert-v01@openssh.com, ecdsa-sha2-nistp521, ecdsa-sha2-nistp521-cert-v01@openssh.com, ecdsa-sha2-nistp384, ecdsa-sha2-nistp384-cert-v01@openssh.com, ecdsa-sha2-nistp256, ecdsa-sha2-nistp256-cert-v01@openssh.com, sk-ecdsa-sha2-nistp256@openssh.com or sk-ecdsa-sha2-nistp256-cert-v01@openssh.com. A Host Key must identify a host distinctly. (1:1 relation).

User roles: Operation, Development, Integration

Outdated and insecure algorithms must not be used. Use the following host key algorithms. Naming may differ from the following list:

Algorithmus	Referenzspezifikationen
ssh-ed448	[RFC-8709]
ssh-ed25519	[RFC-8709]
ssh-ed25519-cert-v01@openssh.com	[RFC-8709]
sk-ssh-ed25519@openssh.com	[RFC-8709]
sk-ssh-ed25519-cert-v01@openssh.com	[RFC-8709]
ecdsa-sha2-nistp521	[RFC-5656]
ecdsa-sha2-nistp521-cert-v01@openssh.com	[RFC-5656]
ecdsa-sha2-nistp384	[RFC-5656]
ecdsa-sha2-nistp384-cert-v01@openssh.com	[RFC-5656]
ecdsa-sha2-nistp256	[RFC-5656]
ecdsa-sha2-nistp256-cert-v01@openssh.com	[RFC-5656]
sk-ecdsa-sha2-nistp256@openssh.com	[RFC-5656]
sk-ecdsa-sha2-nistp256-cert-v01@openssh.com	[RFC-5656]
rsa-sha2-512	[RFC-8332]

rsa-sha2-512-cert-v01@openssh.com	[RFC-8332]
rsa-sha2-256	[RFC-8332]
rsa-sha2-256-cert-v01@openssh.com	[RFC-8332]

Remark on ed448 and ed25519:
ed448 and ed25519 are not (explicitly) recommended by BSI, but no weaknesses are known so far, and therefore they are to be classified as secure.

References:
[RFC-5656] Elliptic Curve Algorithm Integration
[RFC-8709] Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol
[RFC-8332] Use of RSA Keys with SHA-256 and SHA-512 in the Secure Shell (SSH) Protocol

Motivation: An attacker can possibly impersonate a target host undetected.

Implementation example: sshd_config contains:

```
HostKeyAlgorithms ssh-ed448,ssh-ed25519,ssh-ed25519-cert-
v01@openssh.com,sk-ssh-ed25519@openssh.com,sk-ssh-ed25519-cert-
v01@openssh.com,ecdsa-sha2-nistp521,ecdsa-sha2-nistp521-cert-
v01@openssh.com,ecdsa-sha2-nistp384,ecdsa-sha2-nistp384-cert-
v01@openssh.com,ecdsa-sha2-nistp256,ecdsa-sha2-nistp256-cert-
v01@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com,sk-ecdsa-
sha2-nistp256-cert-v01@openssh.com
```

Compliance check: $ sshd -T | grep -i HostKeyAlgorithms

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data

ID: 3.50-51/8.0

Req 7	SSH logging must be enabled.

Logging for SSH must be enabled. It is recommended to use level INFO to get important information but not to get a lot of useless events. If needed higher levels like VERBOSE can also be used.

Motivation: Logging security-relevant events is a basic requirement for detecting ongoing attacks as well as attacks that have already occurred. This is the only way in which suitable measures can be taken to maintain or restore system security. Logging data could be used as evidence to take legal steps.

Implementation example: `sshd_config` contains:

```
SyslogFacility AUTH
LogLevel INFO
```

Compliance check:

```
$ sshd -T | grep -i 'LogLevel\|SyslogFacility'
```

For this requirement the following threats are relevant:
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-7/5.0

Req 8	SSH LoginGraceTime must be set to 120s or less. (OpenSSH default)

The LoginGraceTime parameter restricts the time window for a successful authentication. The longer this period is the

more open unauthenticated connections can be established. Required is a maximum of 120s, this is OpenSSH default setting.

Motivation: An adequate time for LoginGraceTime parameter protects the system against unauthenticated SSH connections which waste system resources.

Implementation example: `sshd_config` contains:

```
LoginGraceTime 120
```

Compliance check:

```
$ sshd -T | grep -i LoginGraceTime
```

For this requirement the following threats are relevant:
- Disruption of availability

ID: 3.04-8/5.0

Req 9	SSH MaxAuthTries must be set to 6 or less. (OpenSSH default)

The MaxAuthTries parameter specifies the maximum number of authentication attempts permitted per connection. This value must be limited to 6 or less attempts, this is OpenSSH default.

Motivation: This parameter will minimize the risk of successful brute force attacks to the SSH server.

Implementation example: `sshd_config` contains:

```
MaxAuthTries 6
```

Compliance check:

```
$ sshd -T | grep -i MaxAuthTries
```

For this requirement the following threats are relevant:
- Disruption of availability

ID: 3.04-9/5.0

Req 10	SSH root login must be disabled.

Unique, personal user accounts are mandatory. Constantly working as root is not permitted. To avoid remote login with user root the login over SSH must be disabled.

Note: It is also possible to achieve an adequate security level if only functional user accounts are used on a system. It must be guaranteed to share SSH keys over a central account management system (e.g. ZAM) for the root user and to enroll them with a configuration management system. Additionally, access must be done over a jump host with personalized accounts. The use of SSH keys for authentication is still mandatory (login with password over SSH is not allowed).

Motivation: Usage of root user for administrative tasks as daily routine is a risk, because root has unlimited rights, exists by default and therefore is an easy target for attacks. For everyday work individual, restricted accounts reduce risk. ("Least Privilege Principle")

Implementation example: `sshd_config` contains:

```
PermitRootLogin no
```

Compliance check:

```
$ sshd -T | grep -i PermitRootLogin
```

For this requirement the following threats are relevant:
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-10/5.0

Req 11	SSH strict mode must be enabled.

SSH StrictModes must be enabled. This enables checks to ensure that SSH files and directories have the proper permissions and ownerships of the login user before allowing an SSH session to open.

Motivation: Prevents usage of folder or files regarding OpenSSH with too open access rights.

Implementation example: `sshd_config` contains:

```
StrictModes yes
```

Compliance check:

```
$ sshd -T | grep -i StrictModes
```

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-11/5.0

Req 12	SSH user authentication must be done with public keys.

Authentication with public/private key must be used for SSH login.
Note: The private key of human beings must be protected with a passphrase.

Motivation: Passwords are usually attackable via Phishing, Keylogger and Brute Force attacks. Generally passwords are easier to attack than private SSH keys, esspecially if passphrases are used for private SSH keys.

Implementation example: `sshd_config` contains:

```
AuthenticationMethods publickey
PubkeyAuthentication yes
```

Compliance check:

```
$ sshd -T | grep -i
'AuthenticationMethods\|PubkeyAuthentication'
```

For this requirement the following threats are relevant:
- Disruption of availability
- Denial of executed activities
- Unnoticeable feasible attacks
- Attacks motivated and facilitated by information disclosure or visible security weaknesses

ID: 3.04-12/5.0

Req 13	SSH password authentication must be disabled.

The login must be done with public/key authentication. Login with password only must be disabled for SSH.

For newer Linux Distribution "`UsePAM yes`" can be necessesary to keep support and system working as expected. In this case PAM configuration must not bypass mandatory user authentication, regardless if it be single factor authentication for unprivileged user or be it 2FA for administrators.

Systems that are connected to a central IAM may require authentication by account name and password. In this case, use is only allowed for centrally managed accounts. This deviation can be realized, for example, by means of "`Match` " configuration, see "Match Introduces a conditional block". Privileged accounts of a central IAM can then implement 2FA authentication, for example through "`AuthenticationMethods password,publickey`".

Motivation: Passwords are usually attackable via Phishing, Keylogger and Brute Force attacks. Generally passwords are easier to attack than private SSH keys, esspecially if passphrases are used for private SSH keys.

Implementation example: `sshd_config` contains:

```
PasswordAuthentication no
KbdInteractiveAuthentication no
ChallengeResponseAuthentication no
UsePAM no
```

Compliance check:

```
$ sshd -T | grep -i
'PasswordAuthentication\|KbdInteractiveAuthentication\|Challenge
ResponseAuthentication\|UsePAM'
```

For this requirement the following threats are relevant:
- Disruption of availability
- Denial of executed activities
- Unnoticeable feasible attacks
- Attacks motivated and facilitated by information disclosure or visible security weaknesses

ID: 3.04-13/5.0

Req 14	SSH IgnoreRhosts must be enabled.

The `.rhosts` is legacy mechanism from the earlier `rlogin` and `rsh` protocols, which allow users to specify hosts and users that are trusted and allowed to log in without supplying a password. This mechanism are considered weak from a security standpoint because they rely on IP addresses and hostnames for authentication, which can be spoofed. Therefor itmust be disabled.

Motivation: Usage of ".rhosts" file would allow login without a authentication feature like SSH private key.

Implementation example: `sshd_config` contains:

```
IgnoreRhosts yes
```

Compliance check:

```
$ sshd -T | grep -i ignoreRhosts
```

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-14/5.0

Req 15	SSH HostbasedAuthentication must be disabled.

As with .rhosts, HostbasedAuthentication allows Logins without providing an authentication attribute. Therefor it must be disabled.

Motivation: If a trust relationship is configured with another system a succesfull attack also means compromise of all

other trusted systems.

Implementation example: `sshd_config` contains:

```
HostbasedAuthentication no
```

Compliance check:

```
$ sshd -T | grep -i HostbasedAuthentication
```

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-15/5.0

Req 16	The usage of the SSH service must be restricted to dedicated groups or users.

For easier and more secure system administration it is necessary to use dedicated users or groups (recommended) for SSH.

Motivation: The usage of dedicated users or groups makes use of SSH more secure through reduction of access rights to the necessary minimum. ("Least Privilege Principle")

Implementation example: `sshd_config` contains options like `AllowUsers`, `AllowGroups`, `DenyUsers`, `DenyGroups` to control which user or groups are allowed to login. See also manpage sshd_config.

For this requirement the following threats are relevant:
- Unauthorized access to the system

ID: 3.04-16/5.0

Req 17	SSH MaxSessions must be set to 10 or less. (OpenSSH default)

The MaxSessions parameter specifies the maximum number of open shell, login or subsystem (e.g. sftp) sessions permitted per network connection. This value must be 10 or less, this is OpenSSH default.

Motivation: To protect a system from denial of service due to a large number of concurrent sessions, use the rate limiting function of MaxSessions to protect availability of sshd logins and prevent overwhelming the daemon.

Implementation example: `sshd_config` contains:

```
MaxSessions 10
```

Compliance check:

```
$ sshd -T | grep -i MaxSessions
```

For this requirement the following threats are relevant:
- Disruption of availability

ID: 3.04-17/5.0

Req 18	SSH tunnel devices must be disabled.

SSH can be used to tunnel services. For management service of Linux servers this is typically not used and can be disabled.

Motivation: SSH tunnel feature can be used by an attacker to tunnel traffic to own destinations.

Implementation example: `sshd_config` contains:

```
PermitTunnel no
```

Compliance check:

```
$ sshd -T | grep -i PermitTunnel
```

For this requirement the following threats are relevant:
- Unauthorized use of services or resources

ID: 3.04-18/5.0

Req 19	SSH TCP port forwarding must be disabled.

TCP forwarding can be used to forward TCP connections through SSH. For management service of Linux servers this is typically not used and can be disabled.
Note: This requirement is not valid for Jumphosts !

Motivation: Bypassing firewall rule sets get possible, as TCP Forwading allows to tunnel over a SSH server.

Implementation example: `sshd_config` contains:

```
AllowTcpForwarding no
```

or better

```
DisableForwarding yes
```

Compliance check:

```
$ sshd -T | grep -i 'Forwarding\|Gateway'
```

For this requirement the following threats are relevant:
- Unauthorized use of services or resources

ID: 3.04-19/5.0

Req 20	SSH agent forwarding must be disabled.

SSH agent forwarding allows to forward authentication requests to other systems over SSH. For management service of Linux servers this is typically not used and must be disabled.
Note: This requirement is not valid for Jumphosts!

Motivation: The server-side deactivation blocks the creation of a server-side agent forwarding socket, this socket consequently cannot be misused.

Implementation example: `sshd_config` contains:

```
AllowAgentForwarding no
```

or better

```
DisableForwarding yes
```

Compliance check:

```
$ sshd -T | grep -i 'Forwarding\|Gateway'
```

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-20/5.0

Req 21	SSH gateway ports must be disabled.

SSH Gateway ports specifies whether remote hosts can connect to ports forwarded for the client. For management service of Linux servers this is typically not used and can be disabled.

Motivation: Bypassing firewall rule sets get possible, as GatewayPorts allows to tunnel over a SSH server.

Implementation example: `sshd_config` contains:
```
GatewayPorts no
```
or better
```
DisableForwarding yes
```

Compliance check:
```
$ sshd -T | grep -i 'Forwarding\|Gateway'
```

For this requirement the following threats are relevant:
- Unauthorized use of services or resources

ID: 3.04-21/5.0

Req 22	SSH X11 forwarding must be disabled.

X11 is not used on Linux servers. The forwarding of X11 over SSH must be disabled.

Motivation: If this feature is not used in a controlled manner, it could be a security risk for servers.

Implementation example: `sshd_config` contains:
```
X11Forwarding no
```
or better
```
DisableForwarding yes
```

Compliance check:
```
$ sshd -T | grep -i 'Forwarding\|Gateway'
```

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-22/5.0

Req 23	SSH PermitUserEnvironment must be disabled.

The SSH option `PermitUserEnviroment` specifies if user defined environment variables are processed by sshd. This function must disabled by setting option argument to "`no`".

Motivation: Enabling the processing environment variable may enable users to bypass SSH access restrictions.

Implementation example: `sshd_config` contains:
```
PermitUserEnvironment no
```

Compliance check:
```
$ sshd -T | grep -i PermitUserEnvironment
```

For this requirement the following threats are relevant:
- Unauthorized use of services or resources

ID: 3.04-23/5.0

Req 24	SSH PermitEmptyPasswords must be disabled.

With the "`PermitEmptyPasswords`" option can be configured the SSH server allows login to an account with an empty password. This must not be allowed.

Motivation: If login without a password remotely over SSH is possible unauthorized users can get access to the server.

Implementation example: `sshd_config` contains:
```
PermitEmptyPasswords no
```

Compliance check:
```
$ sshd -T | grep -i PermitEmptyPasswords
```

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources
- Denial of executed activities
- Unnoticeable feasible attacks

ID: 3.04-24/5.0

Req 25	If SFTP is activated, internal server of OpenSSH must be used.

OpenSSH has its own SFTP daemon. If SFTP should be used this function must be enabled and configured in a secure way.

Motivation: It is necessary to use the OpenSSH SFTP daemon to align the security configuration for all SSH based services and not to have different security levels.

Implementation example: `sshd_config` contains:
```
Subsystem sftp internal-sftp
```

Compliance check: `$ sshd -T | grep -i sftp`

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized use of services or resources

ID: 3.04-25/5.0

3. permissions & key material

Req 26	SSH configuration files and SSH private keys must be protected with a mask of `0600` or more restrictive. Public keys need a mask of 0644 or more restrictive.

The `sshd_config` and files under `sshd_config.d/` contains configuration specifications for sshd. sshd usually runs as root and its configuration and key material (`ssh_host_*_key` and `ssh_host_*_key.pub`) must be protected appropriately.

Motivation: Configuration files and key material need to be protected from unauthorized changes by non-privileged users.

Implementation example:

```
$ chown root:root /etc/ssh/sshd_config
$ chmod 600 /etc/ssh/sshd_config
$ chown root:root /etc/ssh/sshd_config.d/*
$ chmod 600 /etc/ssh/sshd_config.d/*
$ find /etc/ssh -xdev -type f -name 'ssh_host_*_key' -exec chown
root:root {} \;
$ find /etc/ssh -xdev -type f -name 'ssh_host_*_key' -exec chmod
600 {} \;
$ find /etc/ssh -xdev -type f -name 'ssh_host_*_key.pub' -exec
chown root:root {} \;
$ find /etc/ssh -xdev -type f -name 'ssh_host_*_key.pub' -exec
chmod 644 {} \;
```

Compliance check:

```
$ find /etc/ssh -exec ls -l {} \;
```

For this requirement the following threats are relevant:
- Unnoticeable feasible attacks
- Attacks motivated and facilitated by information disclosure or visible security weaknesses

ID: 3.04-26/5.0

Req 27	Minimum key length must be ensured. RSA key material must at least 3072 bit. ECDSA key material must be at least 256 Bit.

User roles: Operation, Development, Integration

When using (classical) asymmetric algorithms and methods, at least the following key lengths must be observed ([TR-02102-1]):

- Methods based on the factorization problem such as RSA: 3000 bits
- Methods based on the discrete logarithm problem such as Diffie-Hellman (DH) or Digital Signature Algorithm (DSA): 3000 bits
- Elliptic curve (ECC) based methods such as Elliptic Curve Diffie-Hellman (ECDH), Elliptic Curve Digital Signature Algorithm (ECDSA) or Edwards-Curve Digital Signature Algorithm (EdDSA): 250 bits

Remark on PQC:
If a hybrid implementation of classical and post-quantum cryptography is used, then the classical part must comply to this requirement.

Reference:
[TR-02102-1] BSI-Technical Guideline: Cryptographic Mechanisms - Recommendations and Key Lengths, Version 2025-01

Motivation: The required key lengths are considered safe until at least 2030.

ID: 3.04-28/5.0

Req 28	SSH moduli smaller than 3072 bit must not be used.

The file "/etc/ssh/moduli" contains pre-generated group parameters – named moduli - for Diffie-Hellman. Here are also moduli available that are not long enough to withstand known attacks.

To avoid the use of short values moduli smaller than 3072 bit must be deleted from file " /etc/ssh/moduli ". The "size" column in /etc/ssh/moduli lists the index of the most significant bit (starting from 0), meaning a 3072-bit prime, whose highest set bit is at position 3071, will be listed as 3071.

Reference:
[TR-02102-4] BSI-Technical Guideline: Cryptographic Mechanisms - Recommendations and Key Lengths - Part 4 SSH, Version 2025-01

Motivation: If the DH modulus is too short the key exchange, then it is not protected in an adequate way.

Implementation example:

```
$ awk '$5 >= 3071' /etc/ssh/moduli > /etc/ssh/moduli.safe
$ mv /etc/ssh/moduli.safe /etc/ssh/moduli
```

Compliance check: $ awk '$5 < 3071' < /etc/ssh/moduli

For this requirement the following threats are relevant:
- Unauthorized access to the system
- Unauthorized access or tapping of data
- Unauthorized modification of data
- Unauthorized use of services or resources

ID: 3.50-47/8.0